

Basic ML Workflow:

Machine learning is the process of building systems that can learn patterns from data and make predictions, instead of being explicitly programmed with fixed rules. Every ML project, regardless of how advanced it is, follows a common sequence of steps. This article explains each step of that workflow with definitions and code examples, using a simple classification problem:

Predicting the species of a flower from its measurements.

Dataset:

A dataset is a structured collection of examples used to train and evaluate a model. It is typically organized in tabular form, where each row represents one observation and each column represents one attribute of that observation.

For this article, I will use the Iris dataset which contains measurements of 150 flowers along with their species.

Code and screenshot:

Data

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame
print(df.head())
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	target
0	0
1	0
2	0
3	0
4	0

Features:

Features are the input variables used by the model to make predictions. They represent the measurable properties of each observation. In this dataset, the four measurements: sepal length, sepal width, petal length and petal width are the features. Features are denoted as X.

Features

```
[4] col = df.columns  
    print(col)  
  
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',  
      'petal width (cm)', 'target'],  
      dtype='object')  
  
[5] X = df.drop(columns=['target'])
```

Labels:

A label is the target variable that the model is trying to predict. In a supervised learning problem, every training example must have a known label. Here, the species of the flower is the label denoted as y.

Labels

```
[6] y = df['target']
```

The distinction between features and labels is fundamental to supervised learning: the model learns a mapping from X to y based on labeled examples.

Train/Test Split:

A model must never be evaluated using the same data it was trained on because this does not indicate whether the model has learned generalizable patterns or has simply memorized the training examples. To address this, the dataset is divided into a training set used to fit the model and a test set used exclusively to evaluate its performance on unseen data.

Train/Test Split

```
[7] from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Model Training:

Model training is the process by which an algorithm adjusts its internal parameters based on the training data, in order to minimize the difference between its predictions and the actual labels. Different algorithms achieve this differently, a decision tree learns a sequence of split conditions, while a neural network adjusts internal weights through repeated exposure to data but the underlying objective is the same.

Model Training

```
[8] ✓ Os from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)
```

...

DecisionTreeClassifier

DecisionTreeClassifier(max_depth=3, random_state=42)

Prediction:

Prediction is the process of applying a trained model to new, unseen input data to generate an output. Once a model has been trained, it can be used to predict labels for the test set.

Prediction

```
[9] ✓ Os predictions = model.predict(X_test)
print(predictions[:10])
```

...

[1 0 2 1 1 0 1 2 1 1]

Evaluation:

Evaluation measures how accurate the model's predictions are by comparing them against the true labels of the test set.

Evaluation

```
[10] ✓ Os from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions))
```

...

Accuracy: 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Accuracy represents the percentage of correct predictions. However, accuracy alone can be misleading for imbalanced datasets. For example, if 95% of samples belong to one class, a model that always predicts that class would achieve 95% accuracy despite being practically useless. In such cases, metrics like precision and recall provide a more complete picture.

Overfitting and Underfitting:

Overfitting occurs when a model learns the training data too precisely, including its noise and irrelevant variations, resulting in high training accuracy but poor performance on new, unseen data. The model fails to generalize.

Underfitting occurs when a model is too simple to capture the underlying pattern in the data resulting in poor performance on both the training and test sets.

```
[11]
✓ Os  ▶ train_acc = model.score(X_train, y_train)
      test_acc = model.score(X_test, y_test)

      print(f"Train accuracy: {train_acc:.2f}")
      print(f"Test accuracy: {test_acc:.2f}")

▼ ... Train accuracy: 0.96
    Test accuracy: 1.00
```

References:

<https://towardsdatascience.com/techniques-for-handling-underfitting-and-overfitting-in-machine-learning-348daa2380b9/>

<https://developers.google.com/machine-learning/crash-course/overfitting/regularization>

<https://gateremark.medium.com/understanding-machine-learning-and-its-basic-workflow-1-fe85a412bb21>